

FreeRTOS Architecture

Introduction

FreeRTOS is a lightweight Real-Time Operating System designed for microcontrollers and small embedded systems.

It provides:

- Multitasking
- Scheduling
- Synchronization
- Inter-task Communication
- Memory Management

FreeRTOS is highly portable and supports many processor architectures.

High Level Architecture



Each layer has a specific responsibility.

1. Application Layer

This is the top-most layer.

Contains all user application tasks.

Examples:

LED Task

UART Task

```
ADC Task  
LCD Task  
Sensor Task
```

Application tasks use FreeRTOS APIs to access kernel services.

Example:

```
xTaskCreate()  
xQueueSend()  
xSemaphoreTake()
```

2. FreeRTOS API Layer

Provides APIs used by application tasks.

Examples:

Task APIs

```
xTaskCreate()  
vTaskDelete()  
vTaskSuspend()  
vTaskResume()  
vTaskDelay()
```

Queue APIs

```
xQueueCreate()  
xQueueSend()  
xQueueReceive()
```

Semaphore APIs

```
xSemaphoreCreateBinary()  
xSemaphoreTake()  
xSemaphoreGive()
```

The API layer acts as an interface between application code and the kernel.

3. FreeRTOS Kernel

The kernel is the heart of FreeRTOS.

Responsibilities:

- Task Management
 - Scheduling
 - Context Switching
 - Time Management
 - IPC Management
-

Scheduler

The scheduler determines which task should execute.

FreeRTOS uses:

Priority Based Scheduling

Highest priority ready task gets CPU.

Example:

Task A Priority = 3

Task B Priority = 2

Task C Priority = 1

Execution order:

A → B → C

Context Switching

Context switching occurs when CPU execution moves from one task to another.

Context includes:

- Program Counter
- Stack Pointer
- CPU Registers
- Status Register

Process:

```
Running Task
|
Save Context
|
Scheduler
|
Restore Context
|
Next Task
```

Time Management

The kernel maintains a system tick count.

Used for:

- Delays
- Timeouts
- Task Scheduling

Example:

```
vTaskDelay(100);
```

4. Memory Management

FreeRTOS uses heap memory for dynamic object creation.

Examples:

- Task Control Blocks
- Task Stacks
- Queues
- Semaphores
- Event Groups

heap_1.c

Features:

- Allocation only
- No free operation

Advantages:

- Fast
- Deterministic

Limitation:

Memory cannot be released.

heap_2.c

Features:

- Allocation and free supported
- No memory coalescing

May suffer from fragmentation.

heap_3.c

Uses standard C library:

```
malloc()  
free()
```

heap_4.c

Most commonly used.

Features:

- Allocation
- Free
- Memory Coalescing

Advantages:

- Reduced fragmentation

heap_5.c

Supports multiple memory regions.

Useful when system contains:

Internal SRAM
External SRAM

5. Inter-Task Communication

Tasks communicate using kernel objects.

Queue

Used to transfer data.

xQueueSend()
xQueueReceive()

Semaphore

Used for synchronization.

xSemaphoreTake()
xSemaphoreGive()

Mutex

Used for resource protection.

Supports:

Priority Inheritance

Event Group

Used for event synchronization.

```
xEventGroupSetBits()
xEventGroupWaitBits()
```

Task Notification

Lightweight alternative to semaphores.

```
xTaskNotify()
ulTaskNotifyTake()
```

6. Port Layer

The Port Layer is hardware dependent.

Purpose:

Make FreeRTOS Portable

Each processor architecture has its own port.

Examples:

```
ARM Cortex-M
RISC-V
PIC32
MSP430
```

Responsibilities

Context Switching

Processor-specific register save/restore operations.

Tick Interrupt

Configures:

SysTick Timer

or

TIM6/TIM7

for RTOS tick generation.

Critical Section Management

Implements:

```
taskENTER_CRITICAL()
taskEXIT_CRITICAL()
```

7. Hardware Layer

Contains actual hardware resources.

Examples:

```
CPU
RAM
Flash
GPIO
UART
SPI
I2C
ADC
DMA
```

The FreeRTOS kernel interacts with hardware through the port layer.

Tick Interrupt Operation

```
SysTick Interrupt
|
v
```



```

Tick Counter Increment
    |
    v
Scheduler Check
    |
    v
Context Switch (If Needed)

```

Typically:

1 Tick = 1 ms

Advantages of FreeRTOS Architecture

Modular

Components are separated logically.

Portable

Runs on many processors.

Small Footprint

Suitable for low-memory microcontrollers.

Real-Time Performance

Deterministic scheduling and fast context switching.

Scalable

Can be used in simple and complex embedded systems.

Interview Questions

What are the major layers in FreeRTOS architecture?

- Application Layer
- API Layer
- Kernel

- Port Layer
- Hardware Layer

What is the role of the scheduler?

To select the highest-priority ready task for execution.

Why is the Port Layer required?

To make FreeRTOS portable across different processor architectures.

Which heap implementation is most commonly used?

```
heap_4.c
```

because it supports allocation, deallocation and memory coalescing.

Memory Management in FreeRTOS (heap_4.c)

Introduction

FreeRTOS uses heap memory to dynamically allocate kernel objects.

Examples:

- Tasks
- Task Stacks
- Queues
- Semaphores
- Mutexes
- Event Groups
- Software Timers

The memory manager used by FreeRTOS is selected by including one of the heap implementation files.

```
heap_1.c  
heap_2.c  
heap_3.c  
heap_4.c  
heap_5.c
```

Among these, `heap_4.c` is the most widely used because it supports both allocation and deallocation while minimizing memory fragmentation.

Why Memory Management is Required?

Whenever a kernel object is created, memory must be allocated.

Example:

```
xTaskCreate();
```

Memory required:

- Task Control Block (TCB)
- Task Stack

Similarly:

```
xQueueCreate();
```

allocates memory for:

- Queue Control Structure
- Queue Storage Area

Therefore FreeRTOS requires a heap manager.

Heap Memory Layout

The heap is a dedicated RAM region managed by FreeRTOS.

Configured through:

```
#define configTOTAL_HEAP_SIZE (20 * 1024)
```

Example:

```
Heap Size = 20 KB
```

Memory layout:

```

+-----+
| FreeRTOS Heap |
+-----+
| Free Block    |
| Used Block    |
| Free Block    |
| Used Block    |
+-----+

```

Features of heap_4.c

Dynamic Allocation

Supports runtime memory allocation.

API:

```
pvPortMalloc()
```

Dynamic Deallocation

Supports releasing memory back to heap.

API:

```
vPortFree()
```

Memory Coalescing

Adjacent free blocks are merged automatically.

This helps reduce fragmentation.

Better Memory Utilization

Because free blocks are merged, large allocations remain possible even after multiple allocations and deallocations.

Memory Allocation

API

```
void *pvPortMalloc(  
    size_t xSize);
```

Parameter

Parameter	Description
xSize	Number of bytes required

Return Value

Return Value	Meaning
Pointer	Success
NULL	Allocation Failed

Example

```
uint8_t *buffer;  
  
buffer =  
    pvPortMalloc(100);
```

Allocates:

100 Bytes

from FreeRTOS heap.

Memory Deallocation

API

```
void vPortFree(  
    void *pv);
```

Parameter

Parameter	Description
pv	Pointer previously allocated

Example

```
vPortFree(buffer);
```

Memory becomes available for future allocations.

Memory Coalescing

One major feature of `heap_4.c` is memory coalescing.

Consider:

```
+---+---+---+---+
| A | B | C | D |
+---+---+---+---+
```

Suppose:

```
B Freed
C Freed
```

Result:

```
+---+---+---+---+
| A | B + C | D |
+---+---+---+---+
```

The free blocks are merged automatically.

Fragmentation

What is Fragmentation?

Fragmentation occurs when memory becomes divided into many small free blocks.

Example:

```
[Used]
[Free 20]
[Used]
[Free 15]
[Used]
[Free 10]
```

Total Free:

45 Bytes

But:

Allocation Request = 30 Bytes

may fail because no single block is large enough.

How heap_4.c Handles Fragmentation

When neighboring free blocks exist:

```
[Free 20]
[Free 15]
```

heap_4.c merges them:

```
[Free 35]
```

This process is called:

Coalescing

and significantly reduces fragmentation.

Heap Configuration

Located in:

```
FreeRTOSConfig.h
```

Example:

```
#define configTOTAL_HEAP_SIZE \
    (20 * 1024)
```

Meaning:

```
20 KB heap memory
```

Heap Monitoring APIs

Get Current Free Heap

API

```
size_t xPortGetFreeHeapSize(void);
```

Purpose

Returns currently available heap memory.

Example:

```
printf("%u",
    xPortGetFreeHeapSize());
```

Get Minimum Ever Free Heap

API


```
size_t xPortGetMinimumEverFreeHeapSize(void);
```

Purpose

Returns the lowest amount of free heap ever available since startup.

Useful for:

- Heap sizing
- Memory optimization

Example:

```
printf("%u",
       xPortGetMinimumEverFreeHeapSize());
```

Dynamic Allocation Example

```
TaskHandle_t xTask;

xTaskCreate(
    TaskFunction,
    "Task",
    256,
    NULL,
    1,
    &xTask);
```

Internally allocates:

```
Task Control Block
+
Task Stack
```

using `pvPortMalloc()`.

Advantages of heap_4.c

- Supports Allocation
- Supports Deallocation

- Memory Coalescing
- Reduced Fragmentation
- Good Memory Utilization
- Suitable for Most Applications

Limitations

- Dynamic allocation overhead exists.
- Fragmentation can still occur in extreme cases.
- Allocation time is not perfectly constant.
- Not ideal for very strict safety-critical systems.

Typical Usage

`heap_4.c` is recommended for:

- Industrial Controllers
- IoT Devices
- Consumer Electronics
- Embedded Linux Companion Controllers
- General Embedded Applications

Interview Questions

Why is `heap_4.c` preferred over `heap_2.c`?

Because `heap_4.c` supports memory coalescing, which reduces fragmentation.

Which APIs are used for dynamic memory allocation in FreeRTOS?

```
pvPortMalloc()  
vPortFree()
```

What is memory coalescing?

Combining adjacent free memory blocks into a larger free block.

What is the purpose of `xPortGetFreeHeapSize()`?

Returns currently available heap memory.

What is the purpose of xPortGetMinimumEverFreeHeapSize()?

Returns the minimum free heap available since system startup and helps estimate actual memory usage.

Embedded Bootloaders

What is a Bootloader?

A Bootloader is a small software program that executes immediately after processor reset.

Its primary responsibility is to initialize hardware and load the operating system or application into memory.

Without a bootloader, the operating system cannot start because essential hardware resources are not yet configured.

Why is a Bootloader Required?

After reset:

- CPU is in a basic state.
- RAM is not initialized.
- Clock system may not be configured.
- Peripherals are disabled.
- Operating system is not loaded.

The bootloader prepares the system so that the kernel/application can execute correctly.

Responsibilities of a Bootloader

Hardware Initialization

Initializes:

- CPU
 - RAM (DDR/SRAM)
 - Clock System
 - UART
 - GPIO
-

Load Operating System

Loads:

- Linux Kernel
- RTOS Image
- Bare-metal Application

from:

- NOR Flash
- NAND Flash
- SD Card
- eMMC
- Network

into RAM.

Pass Boot Parameters

Provides information to the kernel such as:

- Memory Size
- Console Device
- Root File System Location

Transfer Control

After loading the image:

```

Bootloader
  |
Jump to Kernel Entry Point
  
```

Execution is handed over to the operating system.

Types of Bootloaders

First Stage Bootloader (FSBL)

Responsibilities:

- Minimal hardware initialization
- Configure RAM
- Load second stage bootloader

Usually very small in size.

Second Stage Bootloader (SSBL)

Responsibilities:

- Complete hardware initialization
- Load Linux Kernel
- Load Device Tree
- Pass Boot Arguments

Example:

U-Boot

Embedded Linux Boot Process

```
Power ON
|
Boot ROM
|
FSBL
|
U-Boot
|
Linux Kernel
|
Root File System
|
init/systemd
|
Applications
```

Common Embedded Bootloaders

U-Boot

Most widely used embedded Linux bootloader.

Features:

- Command Shell
 - Flash Programming
 - Network Boot
 - Kernel Loading
 - Device Tree Support
-

Barebox

Linux-oriented bootloader with advanced scripting support.

RedBoot

Embedded bootloader based on eCos.

Little Kernel (LK)

Lightweight bootloader commonly used in mobile and embedded devices.

U-Boot Commands

Display environment:

```
printenv
```

Display memory information:

```
bdinfo
```

Boot kernel:

```
bootm
```

Load image:

```
load
```

Advantages of Bootloaders

- Hardware Initialization
- Firmware Upgrade Support
- Debugging Support
- Flexible Boot Sources
- Network Boot Capability

Interview Questions

What is a Bootloader?

A bootloader is the first software executed after reset that initializes hardware and loads the operating system.

Why is a Bootloader Required?

Because hardware resources such as RAM and clocks must be initialized before the operating system can execute.

What is the most popular Embedded Linux Bootloader?

U-Boot

What is the difference between FSBL and SSBL?

FSBL performs basic initialization and loads SSBL, while SSBL performs complete initialization and loads the operating system.

Device Tree in Embedded Linux

What is Device Tree?

A Device Tree is a data structure used to describe the hardware present on an embedded system.

It provides hardware information to the Linux kernel during boot.

The kernel uses this information to initialize drivers and peripherals correctly.

Device Tree is heavily used in ARM-based Embedded Linux systems.

Why Device Tree is Required?

In older Linux systems, hardware information was hardcoded inside the kernel.

Example:

UART Address
GPIO Information

Memory Size
Interrupt Numbers

Whenever hardware changed:

```

Modify Kernel Source
  |
Recompile Kernel
  |
Rebuild Image
  
```

This approach was difficult to maintain.

Device Tree solves this problem by moving hardware information outside the kernel.

Device Tree Concept

```

Kernel
  |
Device Tree
  |
Hardware
  
```

Kernel:

Generic

Device Tree:

Board Specific

This allows the same kernel image to run on multiple boards.

Device Tree Files

Device Tree Source (DTS)

Human-readable source file.

Extension:

```
.dts
```

Example:

```
/ {  
    model = "STM32MP157";  
};
```

Device Tree Blob (DTB)

Compiled binary version of DTS.

Extension:

```
.dtb
```

Generated using:

```
dtc -I dts -O dtb board.dts -o board.dtb
```

Where:

```
dtc = Device Tree Compiler
```

Boot Process Using Device Tree

```
Power ON  
|  
Boot ROM  
|  
Bootloader  
|  
Load Kernel  
|  
Load DTB  
|
```

```
Pass DTB Address
|
Kernel Starts
|
Drivers Initialized
```

Responsibilities of Device Tree

The Device Tree describes:

CPU Information

Example:

```
Number of Cores
CPU Type
Clock Information
```

Memory Information

Example:

```
RAM Start Address
RAM Size
```

Peripheral Information

Examples:

```
UART
SPI
I2C
GPIO
ADC
PWM
USB
Ethernet
```

Interrupt Information

Example:

```
IRQ Numbers
Interrupt Controller
```

Clock Information

Example:

```
Peripheral Clocks
Clock Sources
```

Device Tree Structure

A Device Tree is organized as a hierarchical tree.

Example:

```
/ {
    model = "MyBoard";

    cpus {
    };

    memory {
    };

    soc {
    };
};
```

Root node:

```
/
```

represents the entire board.

Common Nodes

Root Node

```
/ {  
};
```

Represents complete hardware platform.

CPU Node

```
cpus {  
};
```

Describes:

- CPU type
 - Number of CPUs
 - Clock information
-

Memory Node

```
memory {  
};
```

Describes:

- RAM address
- RAM size

Example:

```
memory@c0000000 {  
    reg = <0xc0000000 0x20000000>;  
};
```

Chosen Node

```
chosen {  
};
```

Contains boot-time parameters.

Example:

```
chosen {  
    bootargs = "console=ttyS0";  
};
```

Alias Node

Provides short names.

Example:

```
aliases {  
    serial0 = &uart1;  
};
```

Peripheral Nodes

UART Node

Example:

```
uart1 {  
    status = "okay";  
};
```

Meaning:

Enable UART1

SPI Node

Example:

```
spi1 {  
    status = "okay";  
};
```

```
};
```

I2C Node

Example:

```
i2c1 {  
    status = "okay";  
};
```

GPIO Node

Example:

```
gpio1 {  
    status = "okay";  
};
```

Device Tree and Drivers

Driver Matching Process:

```
Device Tree Node  
    |  
compatible Property  
    |  
Kernel Driver
```

Example:

```
uart1 {  
    compatible = "st,stm32-uart";  
};
```

Kernel searches for a driver supporting:

```
st,stm32-uart
```

and binds that driver to the hardware.

Device Tree Compiler

Compile DTS:

```
dtc -I dts -O dtb board.dts -o board.dtb
```

Options:

```
-I Input Format  
-O Output Format
```

Advantages of Device Tree

Hardware Description Outside Kernel

No need to modify kernel source.

Reusable Kernel

One kernel image can support multiple boards.

Easy Hardware Updates

Only DTS file changes.

Faster Development

Driver developers and board developers can work independently.

Better Maintenance

Hardware information remains separate from driver code.

Typical Device Tree Files

Examples:

```
stm32mp157c.dts  
imx6ull.dts  
am335x-boneblack.dts  
raspberrypi.dts
```

Compiled output:

```
stm32mp157c.dtb
```

Interview Questions

What is Device Tree?

A data structure used to describe hardware to the Linux kernel.

Why is Device Tree required?

To separate hardware description from kernel source code.

Difference between DTS and DTB?

DTS:

```
Human Readable Source File
```

DTB:

```
Binary File Used By Kernel
```

Which tool converts DTS to DTB?

```
dtc
```

(Device Tree Compiler)

What is the purpose of the compatible property?

It allows the Linux kernel to match hardware nodes with the correct device driver.

SUNBEAM